The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5

Max Marks: 100

Student Name	Registration Number
Harsh Tiwari	24BCE10120
Slot	**Date of Submission**
A13	30/03/2026

Chosen Software: Git

CONTENTS

Sno.	Title	Pg no
1.	Introduction	3
2.	Part A - Origin and Philosophy A1. The Problem This Software Was Created to Solve A2. The License - What It Actually Says A3. The Ethics of Open Source	3
3.	Part B - Linux Footprint	7
4.	Part C - The FOSS Ecosystem	9
5.	Part D - Open Source vs Proprietary (Critical Analysis)	11

6.	Shell Script Tasks Script 1: System Identity Report Script 2: FOSS Package Inspector Script 3: Disk and Permission Auditor Script 4: Log File Analyzer Script 5: The Open Source Manifesto Generator	12
7.	Conclusion	16
8.	References	17

1. Introduction

Most of the software that runs the modern world, like the servers that run websites, the tools that developers use every day, and the operating systems inside supercomputers and smartphones, was not made by one company and sold for money. It was built in public, shared for free, and improved by everyone. That is what open source is all about.

I chose Git as the subject of this audit because it is a distributed version control system that is now the main tool for making software. Linus Torvalds made Git in 2005 during a time of trouble in the Linux kernel development community. Since then, it has changed the way millions of developers work together around the world.

This report does a structured audit of Git, looking at its history, its license (GPL v2), the philosophical values it stands for, its impact on Linux systems, its role in the larger FOSS ecosystem, and a critical comparison with proprietary alternatives. In addition to this written analysis, I've made five shell scripts that show how to use Linux in real life.

Part A - Origin and Philosophy

A1. The Problem This Software Was Created to Solve

To comprehend the existence of Git, one must revert to 2005 and grasp the crisis that led to its inception.
Before Git, the Linux kernel, which is one of the biggest and most complicated open-source projects ever, was managed with a proprietary version control system called BitKeeper. BitMover Inc. made BitKeeper, and in 2002, they made it available for free to open-source projects under a special license. Linus Torvalds and the Linux kernel community were very excited about it.
In April 2005, things changed very quickly. Andrew Tridgell, a well-known open-source developer, started to reverse-engineer the BitKeeper protocol. The CEO of BitMover thought this was against the rules and took away the Linux kernel project's free license.

The frustration was immediate and deep. Suddenly, the biggest open-source project in the world didn't have a way to manage its own code. They had problems with proprietary software. Not because it wasn't technically good enough, but because one company owned the license and had its own goals.
Linus Torvalds did not respond by begging or trying to work something out. He built his own, which is what any true open source supporter would do.
Torvalds sent a now-famous email to the Linux kernel mailing list on April 6, 2005, to tell everyone about a new project: "I'm writing some scripts to try to track kernel history, but I suspect I will end up writing a whole SCM thing."

After that, there was a furious two-month development sprint. Torvalds gave himself three design goals:

1. Distributed development: Every developer should have a complete copy of the whole history.
2. Speed: It had to be fast enough to handle the huge Linux kernel repository.
3. Data integrity: The history must be cryptographically safe from damage.

Git was in charge of the Linux kernel source code by June 2005. It came out in July with a GPL v2 license. Linus Torvalds thought that the tools used to make open-source software should also be open source. That's why Git exists.

A2. The License - What It Actually Says

Git is licensed under the **GNU General Public License version 2 (GPL v2)** . To understand this, I obtained the full license text from the official Git repository.

The Four Freedoms of Free Software

Freedom	Description	Does Git Grant It?
Freedom 0	The freedom to run the program for any purpose	Yes
Freedom 1	The freedom to study how the program works and change it	Yes
Freedom 2	The freedom to redistribute copies	Yes
Freedom 3	The freedom to distribute copies of modified versions	Yes

Git's GPL v2 license grants all four freedoms. The preamble states: "When we speak of free software, we are referring to freedom, not price."

Selling Modified Versions: The GPL Question

Can a company take Git, modify it, and sell it without sharing their changes?

Under GPL v2, the answer is **no** — at least not if they distribute the software to others. Section 2(b) requires that if you distribute a modified version, you must also distribute the complete

corresponding source code under the same license terms. This is the "copyleft" provision that ensures the software remains free.

GPL vs MIT

The difference between GPL and MIT licensing lies in what happens when you distribute modified versions.

The **MIT license** is permissive: you can take MIT-licensed code, modify it, and distribute it as part of a proprietary product without sharing your modifications.

The **GPL** is copyleft: if you distribute a modified version, you must also distribute the source code under the GPL.

If I were building something I wanted to remain free for all users forever, I would choose the **GPL**. This aligns with Richard Stallman's vision of software freedom as an ethical imperative.

Free Beer vs Free Speech

The difference between "free as in free beer" and "free as in freedom" is fundamental.

- **Free as in free beer (gratis)** : This refers to price.
- **Free as in freedom (libre)** : This refers to liberty.

Git is both gratis and libre. When you download Git, you pay nothing — free beer. But more importantly, you can fork it, modify it, and create your own version control system based on Git's code — that's free speech.

A3. The Ethics of Open Source

Should All Software Be Free to Use?

Argument for: Open source is all about being open and working together. Anyone can look at open software to find security holes, learn from it, and make it better. Open source lets people check things on their own in critical infrastructure like voting systems and medical devices.

Argument against: Software development requires resources. The proprietary model gives people a

reason to come up with new ideas. If competitors can quickly copy their work, a small business might not be able to stay in business.

My opinion: A mix is good for you. Open source should be the main type of infrastructure and development tools. Proprietary software can exist in environments where incentives for innovation necessitate diverse models.

Is it okay for businesses to use free community labor to grow?

Linux is used a lot by companies like Google and Meta. Is this right?

The moral framework is based on freedom and mutual respect. No one has to give. The GPL says that companies must share changes they make if they give them out. A lot of companies give back—Google made Kubernetes, and Meta made React—not because they have to, but because it makes sense for them.

Standing on the Shoulders of Giants

When I write a simple Python script, I am:

- The Python interpreter, which was made by hundreds of people
- Thousands of people made the Linux kernel.
- The GNU project made the GNU tools
- The C compiler came from the work of Dennis Ritchie
- And so on and so forth

Open source makes this shared inheritance clear and easy to get to.

Does Open Source Help or Hurt New Ideas?

Open source lets people come up with new ideas because they don't have to keep making the same things over and over again. I don't have to make my own web server or database if I want to make a new web app. I can focus on the part that is really new.

The most important new ideas often come from putting existing tools together in new ways. Open standards made the web possible, Linux made cloud computing possible, and Git started the social coding revolution.

Part B - Linux Footprint

To understand how Git lives on a Linux system, I installed it on Ubuntu 22.04.

Installation Method

Git is available through multiple methods:

Via package manager (apt):

```bash
sudo apt update
sudo apt install git
```

Via source compilation:

```bash
wget https://github.com/git/git/archive/refs/tags/v2.40.0.tar.gz
tar -xzf v2.40.0.tar.gz
cd git-2.40.0
make configure
./configure --prefix=/usr/local
make all
sudo make install
```

Distribution support:

- **RPM-based** (Fedora, RHEL): `sudo dnf install git`
- **Debian/Ubuntu**: `sudo apt install git`
- **Arch**: `sudo pacman -S git`

Key Directories

Directory	Purpose
`/usr/bin/git`	Main Git executable
`/etc/gitconfig`	System-wide Git configuration

Directory	Purpose
`~/.gitconfig`	User-specific Git configuration
`/usr/share/doc/git/`	Documentation
`/usr/share/man/man1/git*`	Manual pages

Verification commands:

```bash
which git        # /usr/bin/git
git --version    # git version 2.34.1
```

User and Group

Git runs as a user-level tool. When you run Git commands, they execute with your user privileges:

```bash
whoami           # student
git status       # runs as 'student'
```

This follows the principle of least privilege — Git cannot access files your user cannot access.

Service Management

Git is not typically managed as a service. For Git server components:

```bash
# Git over SSH is managed through SSH
sudo systemctl status sshd

# Git daemon (if configured)
git daemon --base-path=/srv/git --export-all
```

Update Model

Updates reach users through:

1. **Distribution package updates**: `sudo apt update && sudo apt upgrade git`
2. **Security announcements** via mailing lists
3. **Release tarballs** on kernel.org with signed tags

Part C - The FOSS Ecosystem

Dependencies

Dependency	Purpose	License
C library	System calls	LGPL / MIT
zlib	Compression	Zlib
libcurl	HTTP transport	MIT-like
OpenSSL	Cryptography	OpenSSL
Perl	Add-on scripts	GPL / Artistic

What Git Has Made Possible

1. GitHub, GitLab, and other social coding sites

Git's distributed model made a new kind of teamwork possible. GitHub (2008) added a social layer to Git.

2. The fork as a part

Git made it easy to fork, which changed the power dynamics of open source.

3. Integration and deployment all the time

Git hooks make CI/CD pipelines possible.

4. Code for infrastructure

Git stores Terraform and Ansible configurations.

5. Forks and spin-offs

- **libgit2:** A C implementation that works on different platforms
- **JGit:** an implementation in Java
- **Git LFS:** Storage for big files
- **Git Annex:** Organizing files without looking at their contents

Where Git Fits in the Web Stack

Development layer: Git is used by all web developers to keep track of code changes.

Deployment layer: Git hooks start deployments. Git is used to build platforms like Netlify.

Content layer: Git repositories work with static site generators.

Collaboration layer: Git lets teams work together from different places.

Structure of the Community

Mailing lists: git@vger.kernel.org for development and git-security@ for security

Maintainers: Junio C Hamano (lead since 2005) and subsystem maintainers

Governance: a merit-based system with agreement from the community

Conferences: Linux Plumbers Conference, FOSDEM

Companies that hire Git contributors include Google, Microsoft, and GitHub.

Part D - Open Source vs Proprietary

Comparison with **Perforce Helix Core** (proprietary version control):

Dimension	Git (Open Source)	Perforce (Proprietary)
Cost	Free	Expensive per-user licensing
Security	Anyone can audit the code	Only Perforce employees

Dimension	Git (Open Source)	Perforce (Proprietary)
Support	Community, or paid third-party	Official support with SLAs
Freedom to modify	Full freedom under GPL	None
Control	Community-governed	Corporate-controlled
Binary files	Poor (Git LFS helps)	Excellent
Branching	Lightweight, distributed	Centralized, slower
Offline work	Full offline capability	Limited
Learning curve	Steep	Gentler
Ecosystem	Massive	Smaller

The decision

For managing source code, I highly recommend Git. It has a distributed model, branching flexibility, and an ecosystem that make it the best choice for almost all software development.

Perforce's built-in binary handling might make it worth the money for game development or projects with big binary assets.

Would I help Git? Yes. Helping to maintain something that millions of people depend on is what contributing to Git means. Every Git user benefits from even small contributions.

Shell Script Tasks

Script 1: System Identity Report

```bash
#!/bin/bash
# Script 1: System Identity Report

clear
echo "==========================================="
echo "      SYSTEM IDENTITY REPORT"
echo "==========================================="

DISTRO=$(cat /etc/os-release | grep "^PRETTY_NAME" | cut -d'"' -f2)
KERNEL=$(uname -r)
USER=$(whoami)
HOME_DIR=$HOME
UPTIME=$(uptime -p | sed 's/up //')
DATETIME=$(date "+%Y-%m-%d %H:%M:%S")

echo "Linux Distribution : $DISTRO"
echo "Kernel Version     : $KERNEL"
echo "Logged-in User     : $USER"
echo "Home Directory     : $HOME_DIR"
echo "System Uptime      : $UPTIME"
echo "Current Date/Time  : $DATETIME"

echo ""
echo "This system runs on Linux, licensed under GPL v2."
echo "==========================================="
```

Output:

```
==========================================
      SYSTEM IDENTITY REPORT
==========================================
Linux Distribution : Ubuntu 24.04.4 LTS
Kernel Version     : 6.6.87.2-microsoft-standard-WSL2
Logged-in User     : harsh
Home Directory     : /home/harsh
System Uptime      : 2 minutes
Current Date/Time  : 2026-03-30 12:24:58

This system runs on Linux, licensed under GPL v2.
==========================================
harsh@LAPTOP-6C203OKH:/mnt/c/Users/HARSH TIWARI$
```

Concepts: variables, command substitution, echo formatting

Script 2: FOSS Package Inspector

bash

```bash
#!/bin/bash
# Script 2: FOSS Package Inspector

PACKAGE=${1:-git}
echo "Inspecting package: $PACKAGE"

if command -v $PACKAGE &>/dev/null; then
    echo "$PACKAGE is INSTALLED."
    echo "Location: $(which $PACKAGE)"
    $PACKAGE --version 2>/dev/null | head -3
else
    echo "$PACKAGE is NOT installed."
fi

echo ""
case $PACKAGE in
    git)
        echo "Git: Created when proprietary tools failed Linux."
```

```bash
        ;;
    httpd)
        echo "Apache: The server that built the open internet."
        ;;
    mysql)
        echo "MySQL: Open source at the heart of millions of apps."
        ;;
    *)
        echo "Every open source tool represents sharing knowledge."
        ;;
esac
```

Output:

```
Inspecting package: git
git is INSTALLED.
Location: /usr/bin/git
git version 2.43.0

Git: Created when proprietary tools failed Linux.
harsh@LAPTOP-6C203OKH:/mnt/c/Users/HARSH TIWARI$
```

Concepts: command-line args, conditionals, case statements

Script 3: Disk and Permission Auditor

bash

```bash
#!/bin/bash
# Script 3: Disk and Permission Auditor

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "Directory Audit Report"
echo "----------------------"

for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        PERMS=$(ls -ld "$DIR" | awk '{print $1, $3":"$4}')
```

```
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
        printf "%-15s %-30s %-10s\n" "$DIR" "$PERMS" "$SIZE"
    fi
done

echo ""
echo "Git config directory:"
if [ -d "$HOME/.git" ]; then
    ls -ld "$HOME/.git"
fi
```

Output:

```
done

echo ""
echo "Git config directory:"
if [ -d "$HOME/.git" ]; then
    ls -ld "$HOME/.git"
fi
Directory Audit Report
----------------------
/etc            drwxr-xr-x root:root          4.3M
/var/log        drwxrwxr-x root:syslog        25M
/home           drwxr-xr-x root:root          24K
/usr/bin        drwxr-xr-x root:root          95M
/tmp            drwxrwxrwt root:root          24K

Git config directory:
harsh@LAPTOP-6C203OKH:/mnt/c/Users/HARSH TIWARI$
```

Concepts: arrays, for loops, awk parsing, du command

Script 4: Log File Analyzer

bash

```
#!/bin/bash
# Script 4: Log File Analyzer

LOGFILE="$1"
KEYWORD="${2:-error}"
```

```bash
COUNT=0

if [ ! -f "$LOGFILE" ]; then
    echo "Error: File not found."
    exit 1
fi

while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"
echo ""
echo "Last 5 matching lines:"
grep -i "$KEYWORD" "$LOGFILE" | tail -5
```

Output:

```
done < "$LOGFILE"

echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"
echo ""
echo "Last 5 matching lines:"
grep -i "$KEYWORD" "$LOGFILE" | tail -5
Error: File not found.
logout
```

Concepts: while read loop, counters, if-then, tail, grep

Script 5: Open Source Manifesto Generator

bash

```bash
#!/bin/bash
# Script 5: Open Source Manifesto Generator

echo "Answer three questions to generate your manifesto:"
read -p "1. Open-source tool you use daily: " TOOL
read -p "2. What 'freedom' means to you (one word): " FREEDOM
```

```
read -p "3. Something you'd build and share freely: " BUILD

DATE=$(date "+%B %d, %Y")
OUTPUT="manifesto_$(whoami).txt"

cat > "$OUTPUT" << EOF
Manifesto of $(whoami) - $DATE

I use $TOOL every day. It represents software shared, not hoarded.

Freedom means $FREEDOM to me — the freedom to use, study, and share.

If I could build one thing and share it freely, I would build $BUILD.
I stand with open source: code should be transparent, knowledge free.
EOF
echo "Manifesto saved to $OUTPUT"
cat "$OUTPUT"
```

Output:

```
Manifesto of $(whoami) - $DATE

I use $TOOL every day. It represents software shared, not hoarded.

Freedom means $FREEDOM to me — the freedom to use, study, and share.

If I could build one thing and share it freely, I would build $BUILD.
I stand with open source: code should be transparent, knowledge free.
EOF
echo "Manifesto saved to $OUTPUT"
cat "$OUTPUT"
Answer three questions to generate your manifesto:
1. Open-source tool you use daily: git
2. What 'freedom' means to you (one word): Peace
3. Something you'd build and share freely: Projects
Manifesto saved to manifesto_harsh.txt
Manifesto of harsh - March 30, 2026

I use git every day. It represents software shared, not hoarded.

Freedom means Peace to me — the freedom to use, study, and share.

If I could build one thing and share it freely, I would build Projects.
I stand with open source: code should be transparent, knowledge free.
harsh@LAPTOP-6C203OKH:/mnt/c/Users/HARSH TIWARI$
```

Concepts: read for input, string concatenation, file output, date

Conclusion

Git started as a useful way to deal with problems. Linus Torvalds built his own proprietary tool and gave it away when the Linux kernel community lost access to one. This changed the way software is made.

The license analysis showed that GPL v2 is a well-thought-out promise of freedom. The ethical reflections made me think about hard questions about open source. The Linux footprint showed how Git works with system security. The ecosystem mapping showed how things are connected and made new ideas possible. The comparison with Perforce showed that open source isn't always better; it's just different.

My verdict: For real-world software development, I suggest Git. Its freedom isn't just a concept; it leads to better workflows, more new ideas, and more teamwork. And yes, I would give to Git. People who decided to build in the open made every tool I will use. Knowing why they made that choice is not an extra step in technical education; it is the most important part.

References

1. Free Software Foundation. "The Free Software Definition." gnu.org
2. Open Source Initiative. "The Open Source Definition." opensource.org
3. Chacon, Scott. "Pro Git." Apress, 2014.
4. Torvalds, Linus. "Just for Fun." HarperBusiness, 2001.
5. Git Project. https://github.com/git/git
6. GNU General Public License v2.0. gnu.org/licenses/gpl-2.0.html